



BITS Pilani
Pilani Campus

Artificial and Computational Intelligence DSE ZG557

Contact Session 2

Intelligent Agents and Environments
Akhil P V
BITS Pilani WILP

From Acting Rationally to Intelligent Agents



CS1

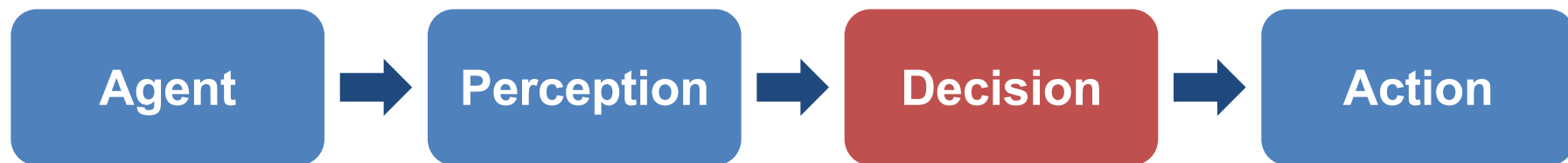
- Acting rationally
- Choosing an action expected to produce the best outcome

CS2

- Agent → Perception → Decision → Action

Today's central question:

- How should we design an agent that acts appropriately in its environment?





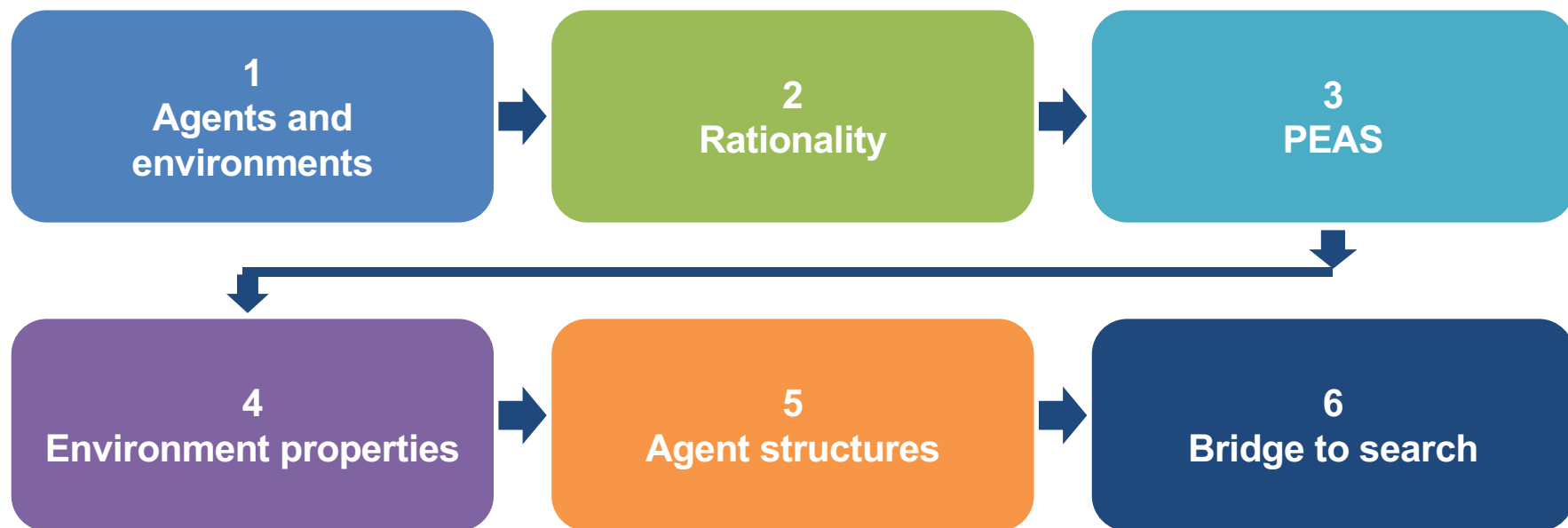
Learning Objectives

By the end of this session, you should be able to:

1. Define an agent, environment, sensor, actuator, percept and percept sequence.
2. Distinguish an agent function, agent program and architecture.
3. Explain rationality and performance measures.
4. Construct a PEAS description.
5. Classify a task environment.
6. Compare major agent-program structures.
7. Recommend an architecture for a given task.

Session Roadmap

Six stages from defining an agent to choosing its architecture.



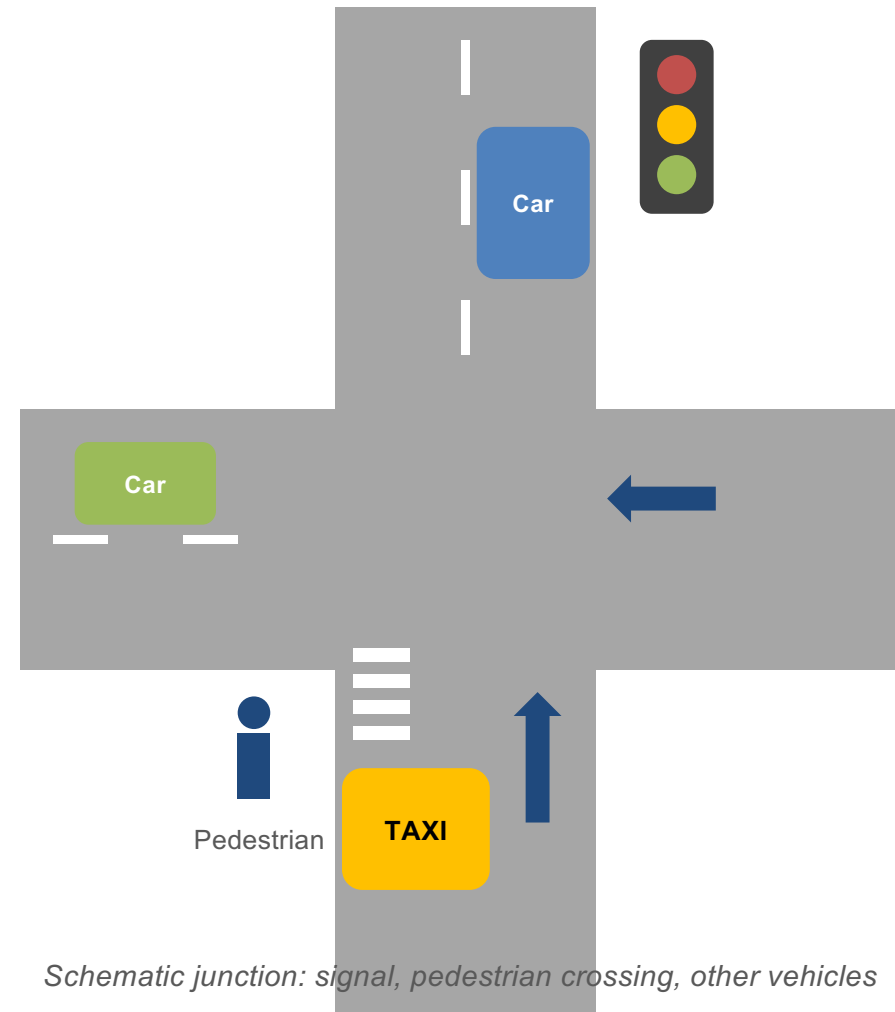
Motivating Scenario: Autonomous Taxi

An autonomous taxi approaches a busy junction. It must decide whether to:

- continue
- slow down
- stop
- change lane
- choose another route

Questions:

1. What information can it perceive?
2. What actions can it perform?
3. What counts as a successful trip?
4. Can it know the result of every action in advance?
5. What internal capabilities does it require?



What Is an Agent?

Agent

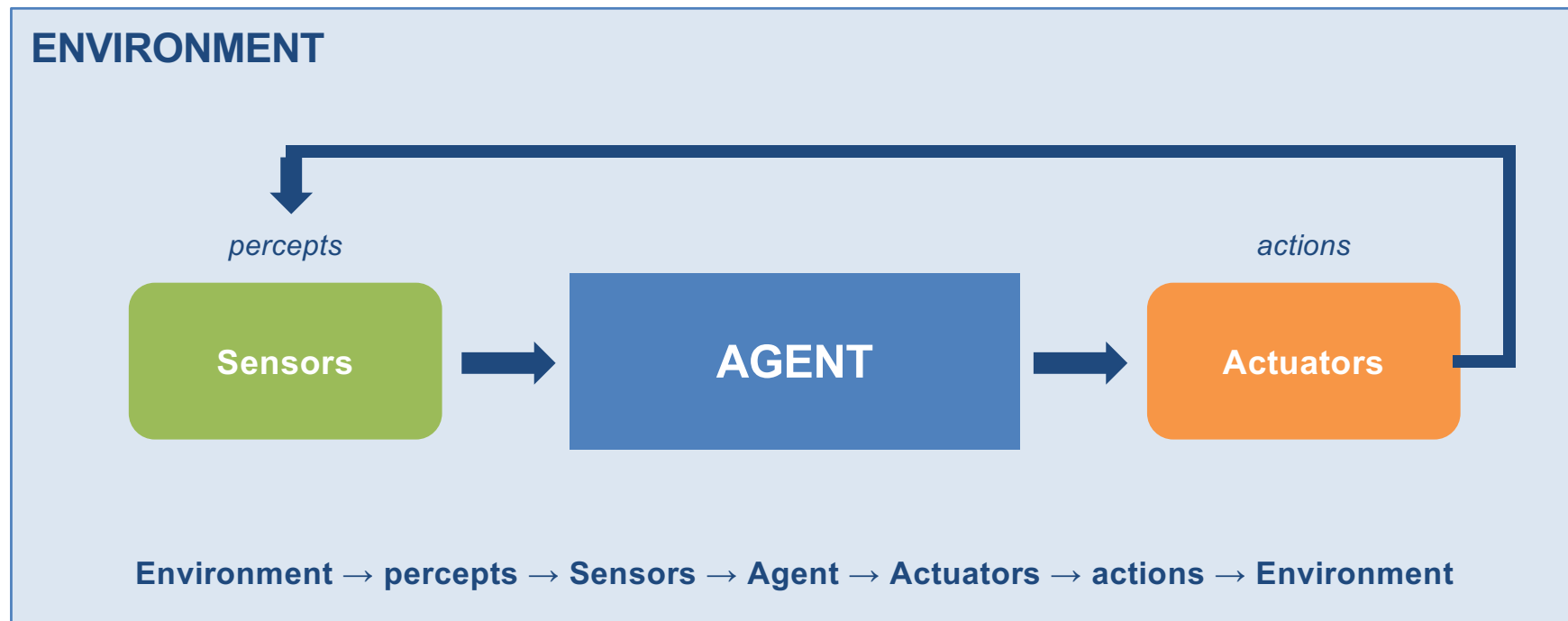
An agent is anything that can be viewed as perceiving an environment through **sensors** and acting upon the environment through **actuators**.

Agent	Sensors	Actuators
Human	eyes, ears, skin	hands, legs, voice
Robot	camera, lidar, encoders	motors, wheels, gripper
Software agent	files, keyboard, network data	display, files, messages, API calls

The Agent–Environment Loop

The loop repeats over time

Perceive → Decide → Act → Perceive again



Percept, Percept Sequence and Action

Percept

Information received by an agent at one instant.

Percept sequence

The complete history of percepts received so far.
 $\langle p_1, p_2, p_3, \dots, p_t \rangle$

Action

The output selected by the agent.

Example: a two-square vacuum world

Time	Percept	Action
t_1	[A, Dirty]	Suck
t_2	[A, Clean]	Right
t_3	[B, Dirty]	Suck

Agent Function

Agent function

An abstract mapping from percept sequences to actions.

$$f: P^* \rightarrow A$$

- P^* = set of possible percept sequences
- A = set of possible actions

Examples

$\langle [A, \text{Dirty}] \rangle \rightarrow \text{Suck}$

$\langle [A, \text{Dirty}], [A, \text{Clean}] \rangle \rightarrow \text{Right}$

Agent Function, Program and Architecture



Concept	Meaning
Agent function	Abstract mapping from percept sequences to actions
Agent program	Concrete implementation that selects actions
Architecture	Computing platform, sensors and actuators
Agent	Architecture running the agent program

Agent = Architecture + Program

Important

The agent program receives the current percept.
If history is required, the program must store it internally.



What Makes an Action Rational?

A rational decision depends on:

- what the agent is trying to achieve
- what it has perceived
- what it already knows
- which actions are available
- what outcomes it expects

Question

Can a rational action still lead to a bad outcome?

Performance Measure

Performance measure

An objective criterion used to evaluate an agent's behaviour.

Agent	Possible performance measures
Vacuum cleaner	cleanliness, time, energy, noise
Taxi agent	safety, legality, speed, comfort, profit
Spam filter	accuracy, low false-positive rate
Medical-diagnosis system	patient health, safety, cost

A rational agent attempts to maximise the expected value of its performance measure.

Reward the Outcome, Not the Wrong Behaviour



An agent optimises exactly the measure it is given.

Poor measure

Reward the vacuum cleaner for every Suck action.

Possible consequences:

- unnecessary Suck actions
- wasted energy
- behaviour optimised for the score rather than cleanliness

Better measure

Evaluate:

- cleanliness over time
- energy consumed
- noise
- unnecessary movement

Measure the outcome you actually want.



Rational Agent: Formal Definition

A rational agent selects, for each percept sequence, the action expected to maximise its performance measure.

Considering:

1. the percept sequence so far
2. prior knowledge of the environment
3. available actions
4. the performance measure

$$a^* = \arg \max_{a \in A} \text{Expected Performance}$$

given percepts, knowledge and action a

Rationality Is Not Omniscience

Rationality is judged on the evidence available at decision time.

Rationality	Omniscience
Uses available evidence	Knows actual future outcomes
Maximises expected performance	Always knows which action will succeed
Can experience an unlucky outcome	Cannot be surprised
Achievable as a design objective	Unrealistic in complex environments

Example

A taxi selects the best route using current data. An unexpected accident later causes delay.

A bad outcome does not necessarily imply an irrational decision.

Information Gathering, Learning and Autonomy



A rational agent may improve its decisions through:

Information gathering

Actions that obtain useful new percepts.

Learning

Improving behaviour using experience.

Autonomy

Depending increasingly on its own experience rather than only on built-in knowledge.

Example: a medical agent requests a test before recommending treatment.

Activity: Is the Decision Rational?

Delivery robot

Route	Expected time	Collision risk
A	10 minutes	20%
B	14 minutes	2%

Performance priorities

- safety: 60%
- delivery speed: 40%

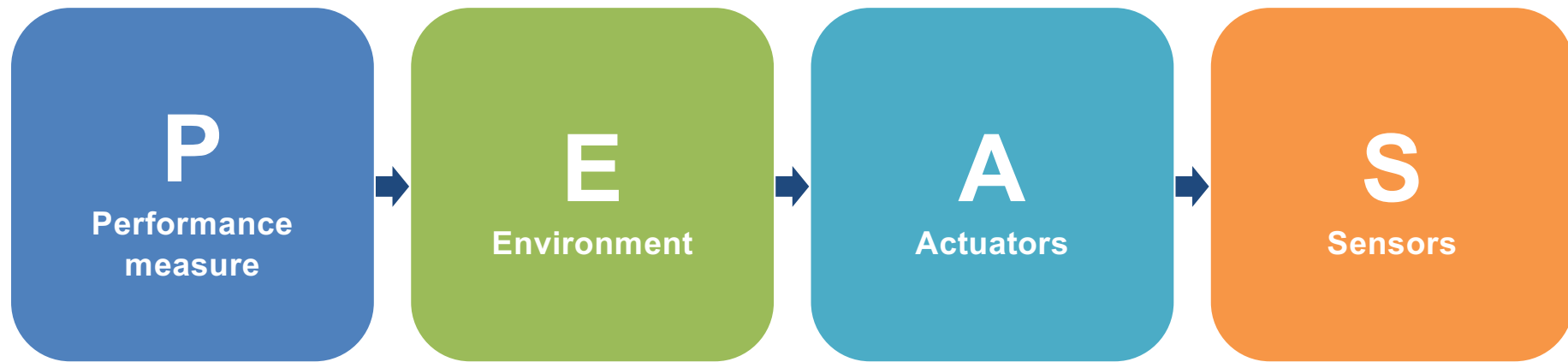
Discuss

1. Which route appears preferable?
2. Is the information sufficient?
3. What additional information is required?

What Is a Task Environment?

A task environment describes the problem faced by an agent.

The first step in agent design is to specify PEAS.



Design questions

- What should be achieved?
- Where does the agent operate?
- How can it act?
- What can it perceive?

PEAS Example: Automated Taxi

A complete task-environment specification for the automated taxi.

PEAS element	Automated taxi
Performance	safety, legality, speed, comfort, profit
Environment	roads, traffic, pedestrians, passengers, signals
Actuators	steering, accelerator, brake, horn, indicators, display
Sensors	cameras, GPS, lidar, speedometer, odometer, engine sensors

Design issue: the objectives may conflict.

PEAS Example: Medical-Diagnosis System



The same framework applies to a purely software agent.

PEAS element	Medical-diagnosis system
Performance	patient health, diagnostic accuracy, safety, reasonable cost
Environment	patient, hospital, medical staff, medical records
Actuators	questions, test recommendations, diagnosis, treatment suggestions, referrals
Sensors	symptoms, findings, patient responses, test results, history

Software systems also have sensors and actuators.

PEAS Activity: Part-Picking Robot

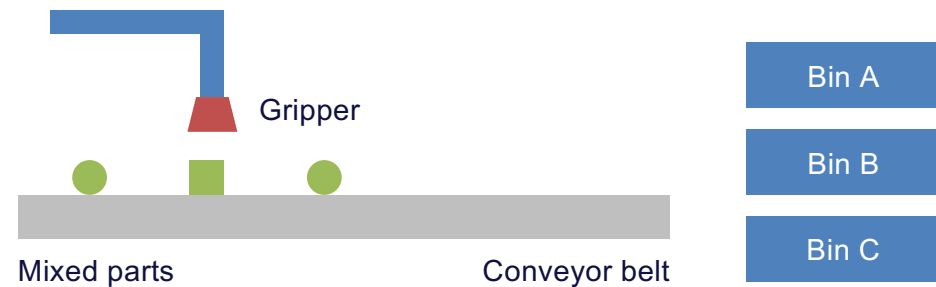


Agent: Part-picking robot

PEAS element	Your answer
Performance measure	?
Environment	?
Actuators	?
Sensors	?

Time: 3 minutes

Robotic arm



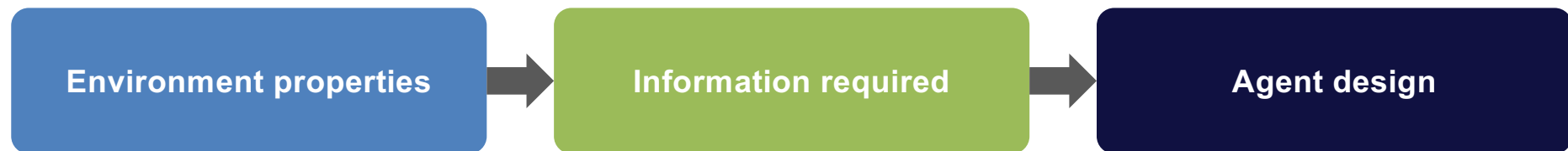
Answer reveal

- **Performance:** percentage of parts placed in the correct bins
- **Environment:** conveyor belt, parts and bins
- **Actuators:** jointed arm and gripper
- **Sensors:** camera and joint-angle sensors

Why Classify the Environment?



The same agent architecture does not work equally well in every environment.



Examples

- Complete information may allow direct rules.
- Missing information may require memory.
- Uncertain outcomes may require probability.
- Changing environments require timely decisions.
- Competing agents require strategic reasoning.

Seven Environment Dimensions



Dimension	Alternatives
Observability	fully observable / partially observable
Number of agents	single-agent / multiagent
Predictability	deterministic / stochastic
Dependence across time	episodic / sequential
Change during deliberation	static / dynamic / semidynamic
State and action values	discrete / continuous
Knowledge of environment rules	known / unknown

T1, §2.3.2 and Chapter 2 summary

Fully Observable vs Partially Observable



Fully observable

The sensors provide all task-relevant information needed for action selection.

Example:

- Chess board



Complete board visible

Partially observable

Some task-relevant information is hidden, missing or noisy.

Examples:

- Poker
- Medical diagnosis
- Taxi driving
- Local-sensing vacuum cleaner



Hidden cards

Important

Fully observable does not mean knowing everything.
It means knowing everything currently relevant to the task.

Single-Agent vs Multiagent

Single-agent

Only one decision-making agent needs to be modelled.

Example: Crossword puzzle

Multiagent

Other agents affect the outcome.

Relationship	Example
Competitive	chess, auction
Cooperative	robot team
Mixed	road traffic

Should every moving object be modelled as an agent?

Deterministic vs Stochastic

Deterministic

Given the current state and action, the next state is completely determined.

$$\text{Result}(s, a) = s'$$

Example:

- Legal move in the 8-puzzle

Stochastic

An action may have several possible outcomes.

$$P(s' | s, a)$$

Examples:

- Vehicle motion on a slippery road
- Medical-treatment response
- Weather-sensitive planning

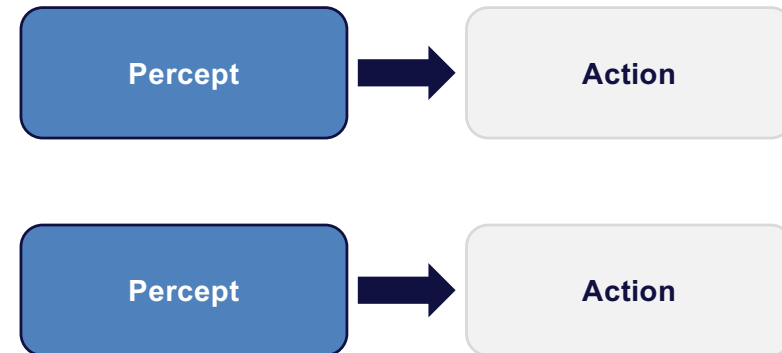
Episodic vs Sequential

Episodic

Each decision can be treated independently.

Examples:

- classify one email
- inspect one manufactured part



Sequential

Current actions affect future states and future decisions.

Examples:

- chess
- robot navigation
- investment planning



Static, Dynamic and Semidynamic



Type	Meaning	Example
Static	Environment does not change while the agent deliberates	crossword
Dynamic	Environment may change during deliberation	taxi driving
Semidynamic	Environment is unchanged, but performance changes with time	chess with a clock

Design consequence

Dynamic environments require timely action.

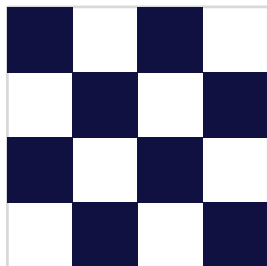
Discrete vs Continuous

Discrete

States, percepts, actions or time steps are finite or countable.

Examples:

- chess moves
- 8-puzzle
- tic-tac-toe



Countable states

Continuous

Variables may take values over continuous ranges.

Examples:

- speed
- steering angle
- temperature
- robot-arm position



Any value in a range

Some environments contain both discrete and continuous elements.

Known vs Unknown

Known environment

The agent knows the rules and the effects or probabilities of actions.

Example: Chess rules supplied to the agent.

Unknown environment

The agent must learn how the environment behaves.

Example: A robot exploring an unfamiliar machine or game.

Do not confuse

Question	Dimension
Can the agent observe the current state?	observable / partially observable
Does the agent know how the world works?	known / unknown

Environment Classification Activity



Classify each task

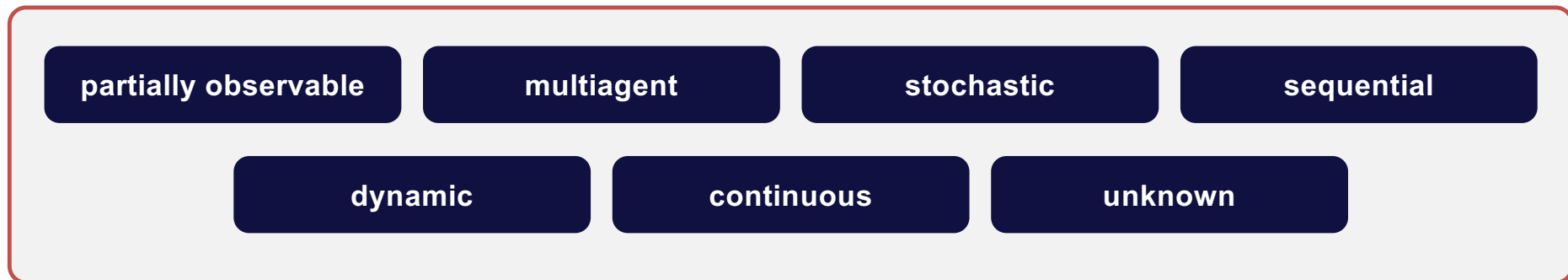
Task	Obs.	Agents	Det./Stoch.	Epis./Seq.	Static/Dyn.	Disc./Cont.	Known?
Chess with clock							
Spam filtering							
Taxi driving							
Medical diagnosis							

Time: 5 minutes

T1, §2.3.2, Figure 2.6

The Hardest Environment

A particularly difficult task environment is:



Such an agent may require

- Internal state
- Prediction
- Goals
- Utility
- Learning
- Rapid action

Example: Autonomous operation in an unfamiliar real-world setting.

From Environment Analysis to Agent Design



Principle: the appropriate agent program depends on the nature of the task environment.

Why Not Use a Giant Lookup Table?



Table-driven agent

Store: percept sequence → action

Conceptual program

Append the new percept to history
Look up the complete percept sequence
Return the stored action

Possible entries over T percepts:

$$\sum_{t=1}^T |P|^t$$

Why it fails

- number of percept histories grows explosively
- table is impossible to construct
- poor generalisation
- no meaningful autonomy
- unsuitable for real environments

Four Basic Agent-Program Designs



Design	Core idea
Simple reflex	React to the current percept
Model-based reflex	Maintain internal state
Goal-based	Choose actions that achieve goals
Utility-based	Choose the most desirable expected outcome

Learning can improve any of these designs.



T1, §2.4, pp. 48–57.

Simple Reflex Agent

Uses the current percept and condition–action rules.

Current percept → Rule match → Action

Example:

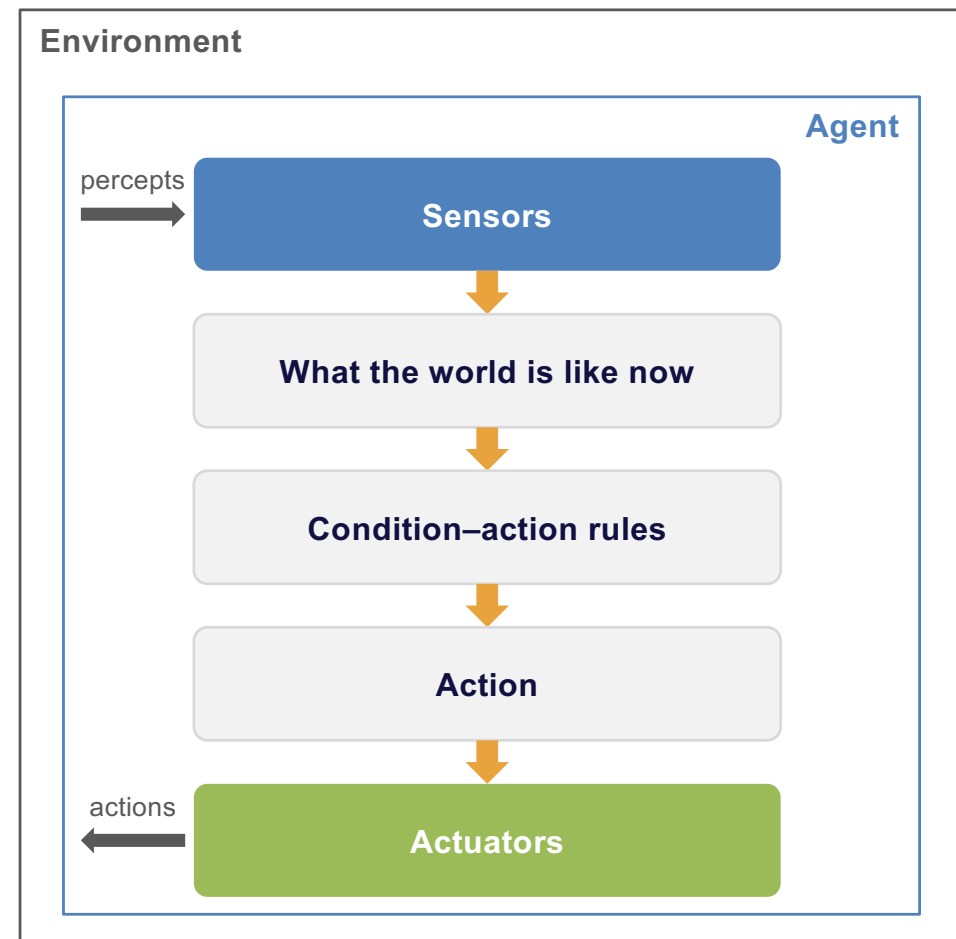
```
IF CurrentSquare = Dirty
THEN Suck
```

Suitable when:

- The environment is fully observable.
- The correct action depends mainly on the current percept.

Limitation:

No memory of unobserved aspects of the world.



Model-Based Reflex Agent

Maintains an internal state representing aspects of the world that are not currently visible.

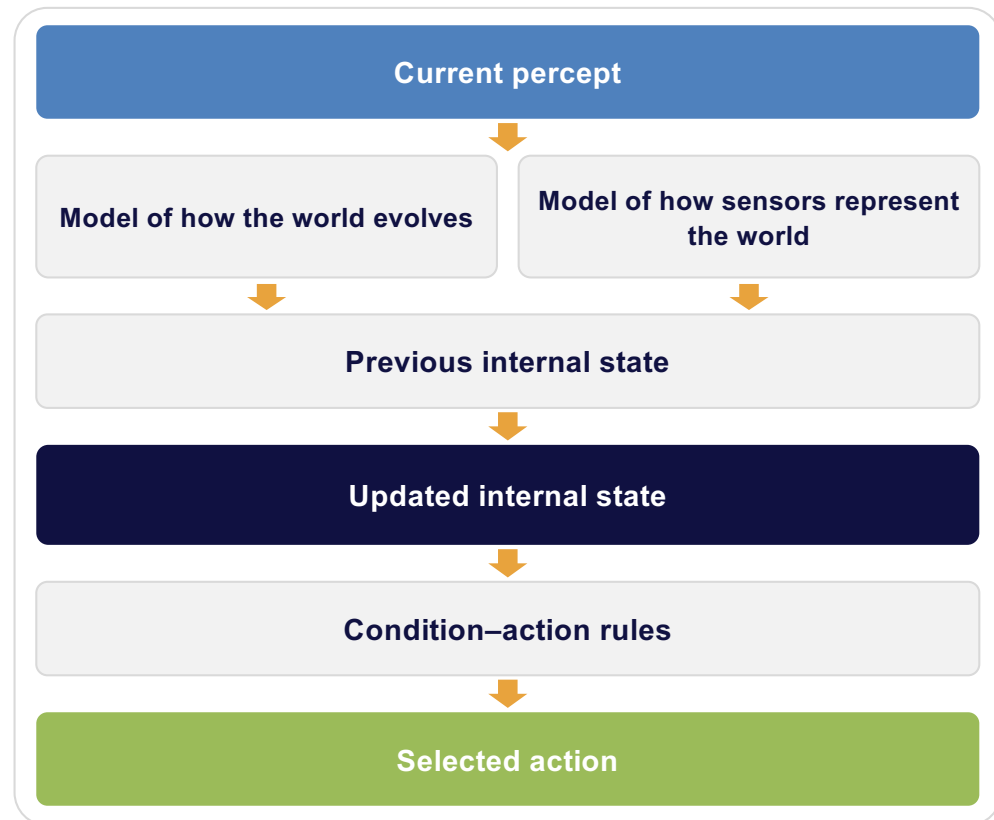
The internal state is updated using:

- the previous internal state
- the previous action taken
- the current percept
- the transition model of the world
- the sensor model of percepts

Percept + Previous state + Model → Updated state → Action

Usefulness:

Handles partially observable environments.



Goal-Based Agent

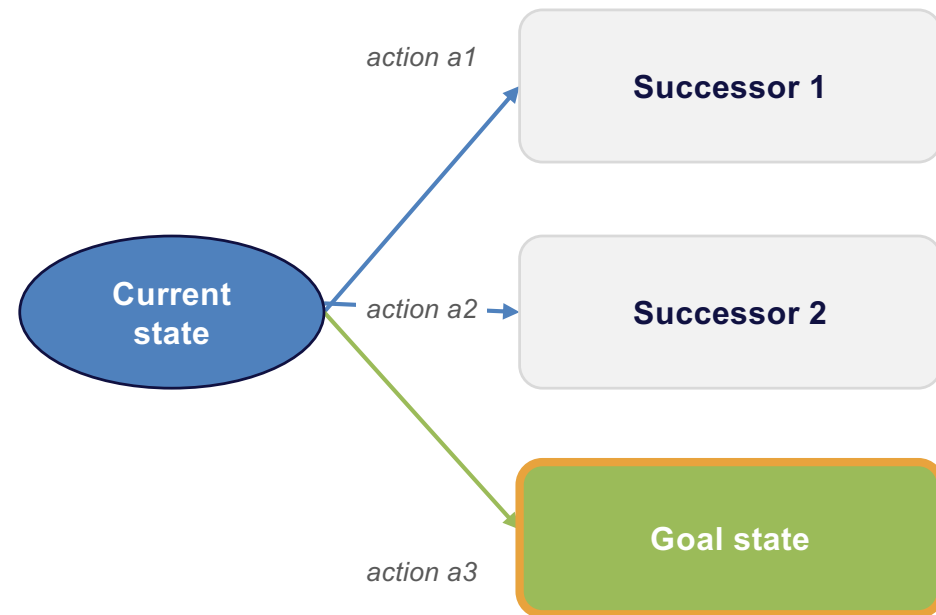
Uses explicit goals to evaluate possible future actions.

Central question: *Which action or sequence of actions can reach the goal?*

Current state + Goal + Predicted consequences → Action

Example:

- **Current state:** In(Arad)
- **Goal:** In(Bucharest)
- **Decision:** Choose a route toward the goal.



Connection: Goal-based behaviour motivates search and planning.

Utility-Based Agent

A goal identifies acceptable outcomes.
Utility ranks their desirability.

$U(s)$ = desirability of state s

Under uncertainty:

a^* = action with the highest expected utility

Comparing three routes

Route	Time	Safety	Comfort
A	best	low	medium
B	medium	high	high
C	poor	high	medium

Why utility helps

Several routes may reach the goal; utility expresses preferences and trade-offs between them.

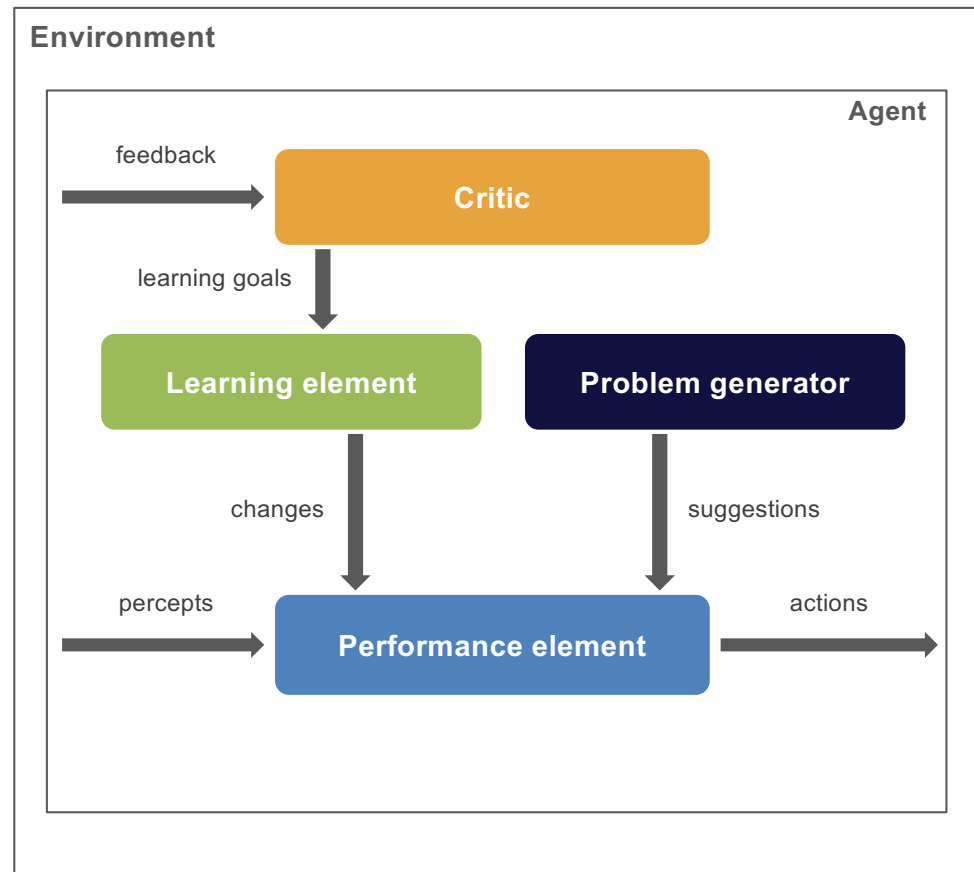
Learning Agent



An agent can improve its performance through experience.

Component	Role
Performance element	selects external actions
Learning element	improves the performance element
Critic	evaluates behaviour using feedback
Problem generator	suggests exploratory actions

Learning can enhance reflex, model-based, goal-based or utility-based agents.



Comparing Agent Designs

Four basic designs, distinguished by memory, goals and preferences.

Agent design	Current percept	Internal state	Goal	Utility	Learning possible
Simple reflex	✓	—	—	—	✓
Model-based reflex	✓	✓	—	—	✓
Goal-based	✓	usually ✓	✓	—	✓
Utility-based	✓	usually ✓	often ✓	✓	✓

Memory rule

Reflex → reacts · Model-based → remembers · Goal-based → aims
Utility-based → optimises · Learning → improves

Architecture Selection Activity

Choose and justify an agent design for the scenario below.

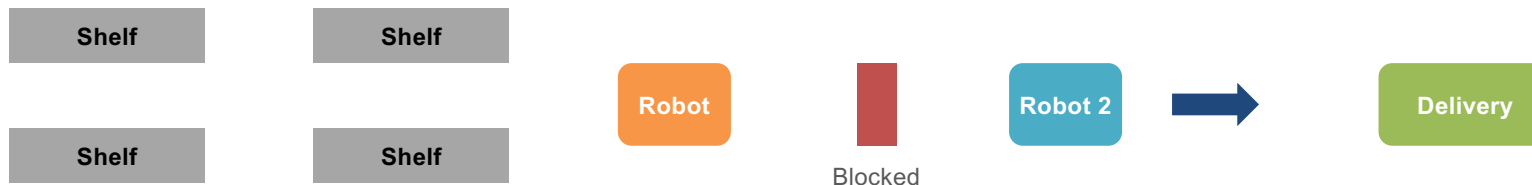
Scenario: warehouse robot

The robot must:

- transport packages to specified locations
- operate when some shelves are temporarily hidden
- share aisles with other robots
- avoid collisions
- balance speed, energy and safety
- adapt when the warehouse layout changes

Discuss

1. Is a simple reflex agent sufficient?
2. Is internal state required?
3. Are explicit goals required?
4. Is utility required?
5. Would learning help?



Application activity based on T1 §§2.3–2.4.

How Should an Agent Represent the World?



A brief extension: three ways of representing states.

Representation	Meaning	Example
Atomic	State treated as an indivisible label	State 27
Factored	State represented using variables and values	Location = A, Dirty = True
Structured	Objects and relationships represented explicitly	Robot carries Box1 in Room2



Increasing expressiveness may also increase reasoning complexity.

Contact Session 2 Summary

Five ideas that together define the agent-design framework.

1

Agent

Perceives through sensors and acts through actuators.

2

Rationality

Maximises expected performance using available evidence.

3

PEAS

Specifies performance, environment, actuators and sensors.

4

Environment properties

Determine difficulty and information requirements.

5

Agent structures

Reflex, model-based, goal-based, utility-based and learning-enhanced.

Task environment → Agent design

Exit Check and Preparation for CS3

Check your understanding, then prepare for the next session.

Exit check

1. How is an agent function different from an agent program?
2. Why is rationality different from omniscience?
3. Give a PEAS description for an interactive English tutor.
4. Why does partial observability motivate internal state?
5. When is utility required in addition to a goal?

Next session

Problem-Solving Agents and Search

Contact Session 3

Required reading

- T1, §§3.1–3.4
- T1, §§3.5.1–3.5.2

Revise

- Queues, stacks and priority queues

Rational agents (CS2) → problem-solving agents and search (CS3)

References



Authoritative sources for Contact Session 2.

Primary course source

Birla Institute of Technology & Science, Pilani
Digital Learning Handout: Artificial and Computational Intelligence
Course No. DSE ZG557, Version 5.0

Prescribed textbook

Stuart Russell and Peter Norvig
Artificial Intelligence: A Modern Approach
Third Edition

Contact Session 2

- §2.1 Agents and Environments
- §2.2 Good Behavior: The Concept of Rationality
- §2.3 The Nature of Environments
- §2.4 The Structure of Agents
- §2.5 Summary and Exercises